Introduction to Data Science

BASIC TRACK · LECTURE 2



Acquiring, storing and preparing data

Data sources, data types, storage formats, tables in pandas and data cleaning

Prof. Dimitar Trajanov, PhD
Prof. Igor Mishkovski, PhD
Prof. Andrea Kulakov, PhD
Prof. Slobodan Kalajdziski, PhD

"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Vezilka
EuroHPC Joint Undertaking
Co-funded by the European Union
Republic of North Macedonia
Ministry of Digital Transformation

Agenda

1. **Data, sources and open data**
   APIs, scraping, open portals
2. **Data types, measurement and quality**
   Variables, levels of measurement, tidy data
3. **Storage formats**
   Row and column storage, CSV and Parquet
4. **Working with tables in pandas**
   Selection, grouping, merging, join cardinality
5. **Cleaning, missing values and outliers**
   Copy-on-Write, MCAR, MAR, MNAR, imputation
6. **Encoding, scaling and the law**
   Categories, dates, standardisation; GDPR, licences

Learning objectives of the lecture

- After this lecture, the student is able to:
  - Compare the ways of acquiring data and judge whether scraping is permitted.
  - Classify variables by **level of measurement**, assess data quality and reshape a table into **tidy** form.
  - Explain why **columnar storage** such as Parquet suits analysis.
  - Select, filter, group and merge tables with **pandas**, and check the **cardinality of a merge**.
  - Clean a table with repeatable column operations under pandas 3 **Copy-on-Write**.
  - Distinguish **MCAR, MAR and MNAR**, choose deletion or imputation, and treat outliers.
  - Encode categorical and date variables and **standardise** numerical features.
  - Apply the GDPR, the LPDP and data licences to the collection of data.

01

Data, sources and open data

Data sources, acquisition channels and reuse metadata establish the basis for subsequent analysis.



Data: definition, sources and the five Vs

- A *datum* is a recorded value or observation; **data are collections of such values**, interpreted using their context and metadata.
- Data originate in **business operations**, in public and social sources, and in sensors connected through the Internet of Things.
- The *five Vs* provide a **descriptive framework** for large-scale data; they are not a formal definition or a size threshold.
  - **Volume**: the amount of data to be stored and processed.
  - **Variety**: the diversity of types and structures.
  - **Velocity**: the rate at which new data arrive.
  - **Veracity**: the degree to which the data can be trusted.
  - **Value**: the use that justifies collecting them.

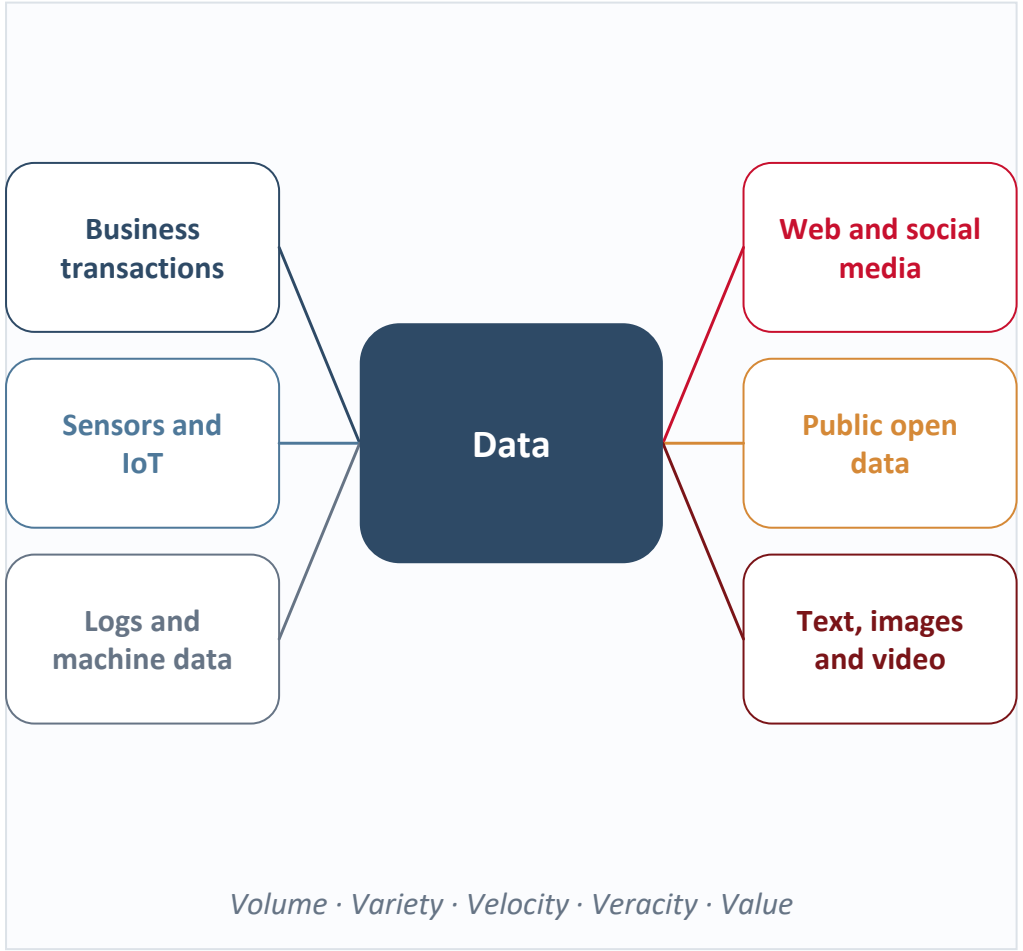
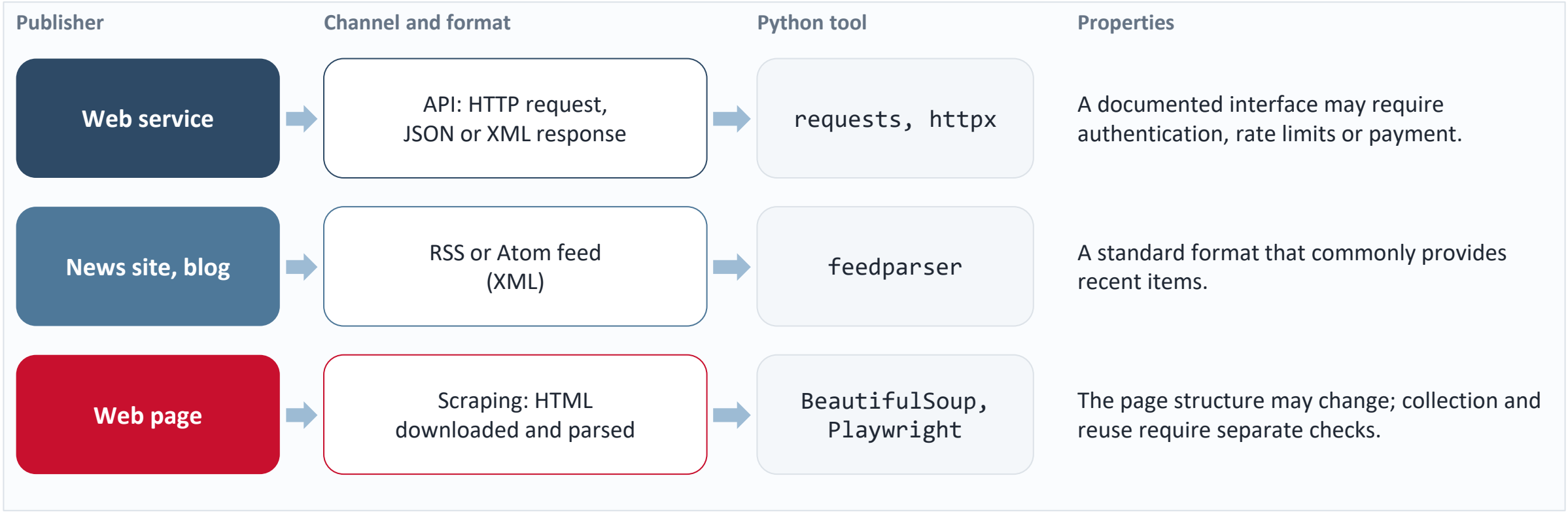


Figure 1 — typical sources of data

Gathering data online: APIs, feeds and web scraping



- An *API* returns structured data under a documented contract, a *feed* publishes recent items in a standard format, and *web scraping* extracts data from pages written for people; an API or a download is **preferred whenever one exists**.

Web scraping: technical, legal and ethical limits

- Scraping is considered when **no suitable API, feed or download** exists; technical accessibility alone does not authorise reuse.
- The file `robots.txt` (RFC 9309, 2022) states which paths automated clients are **requested to avoid**; it is not access authorisation.
- The *terms of service* may forbid automated collection or reuse.
- Collection of **personal data** requires a legal basis under the applicable data-protection law.
- A compliant scraper **limits its request rate**, identifies itself and caches responses; copyright and database rights also apply.
  - The decision is recorded, so that the collection can be **justified later**.

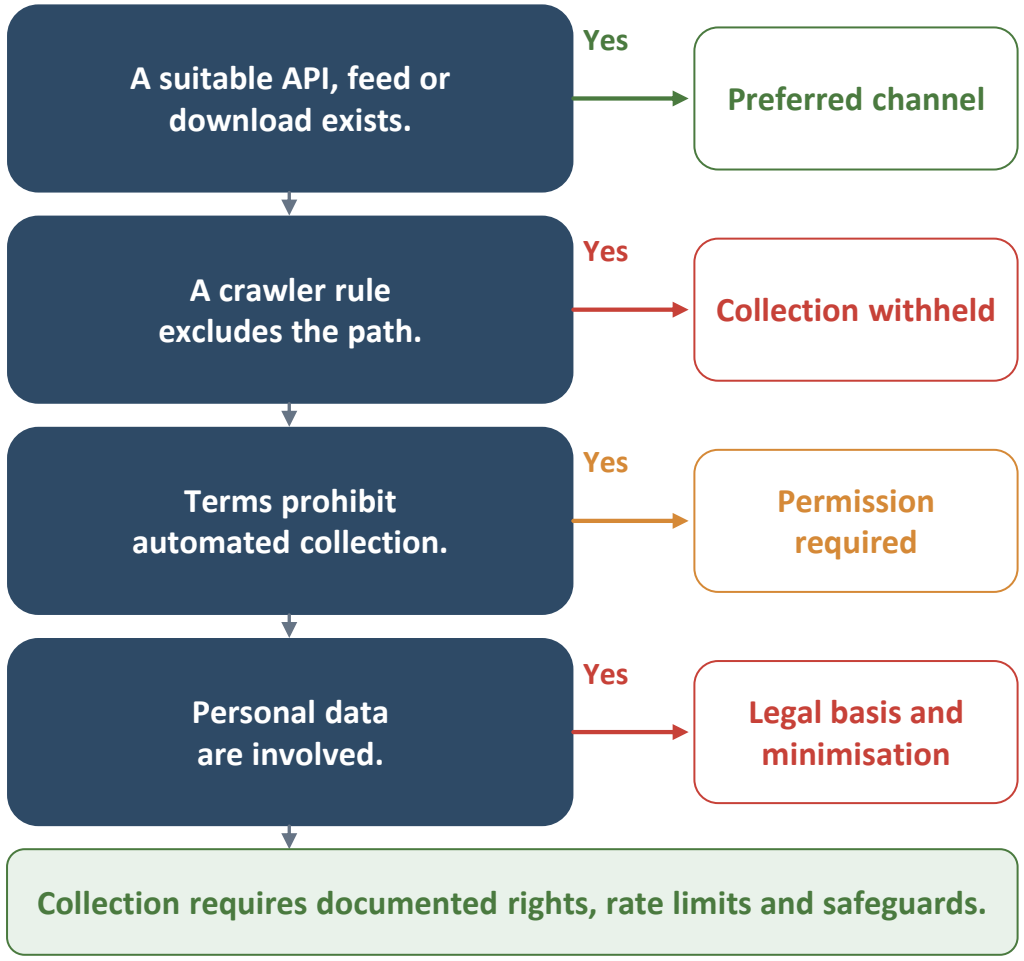


Figure 2 — checks before scraping a site

Open data portals and the formats they publish

- *Open data* permit anyone to access, use, modify and share them, **under an open licence** or applicable public-domain status.
  - The official portal for European public data, **data.europa.eu**, harvests the national portals.
  - The Macedonian national portal, **data.gov.mk**, lists 781 data sets (CKAN API, 4 October 2026).
  - **World Bank Open Data** and **OECD Data** provide indicators by country and year.
  - **Hugging Face** and **Kaggle** host machine-learning data; documentation and reuse terms vary by data set.
- The cached format facets contain many **CSV and spreadsheet entries**; JSON entries are uncommon.
  - Normalised facet counts may overlap across spellings and formats; they are not counts of distinct resources.

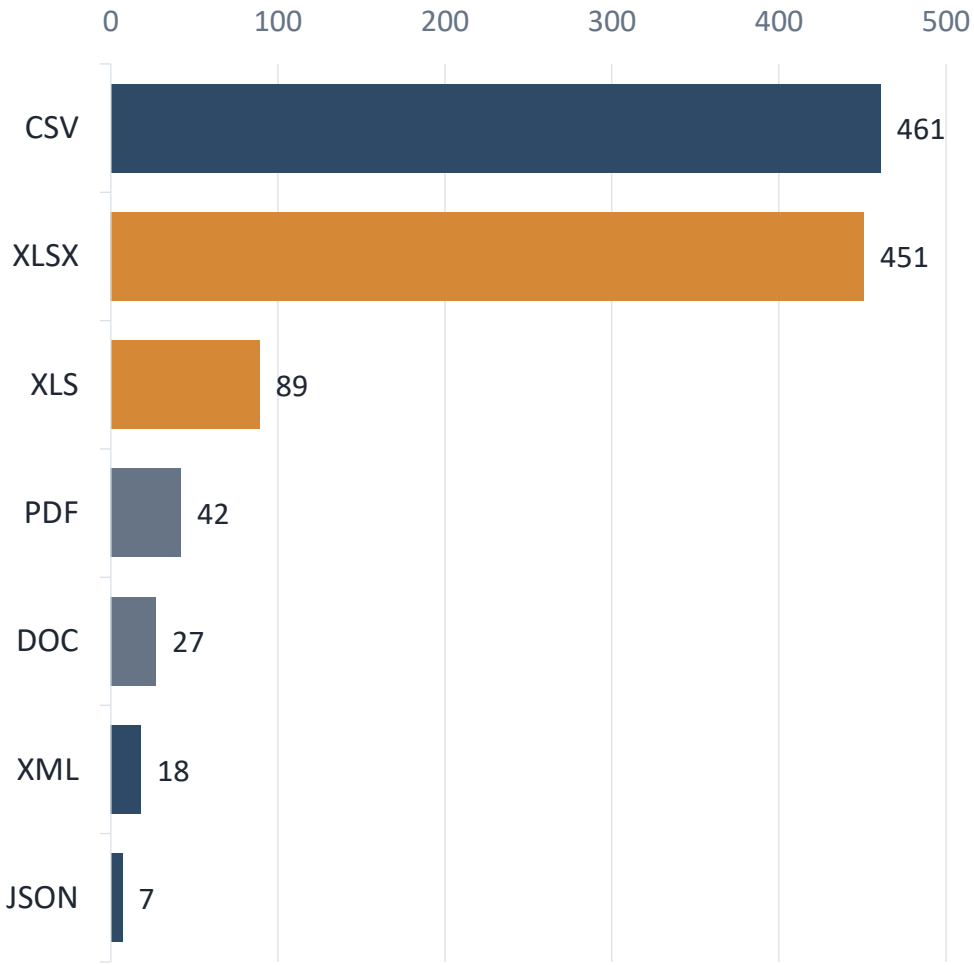


Figure 3 — cached format facets, 4 Oct 2026

02

Data types, measurement and quality

The values inside a data set, the scales on which they are measured, their quality, and the tidy form that analysis expects



Structured, semi-structured and unstructured data

Structured: a table

id	city	temp °C
1	Skopje	2.2
2	Bitola	-0.4
3	Ohrid	3.1
4	Štip	1.8

Every record has the same fields, with one type per column.

Semi-structured: JSON records

```
{"id": 1, "city": "Skopje",
 "temp": 2.2}
{"id": 2, "city": "Bitola",
 "temp": -0.4,
 "station": {"alt_m": 586}}
{"id": 3, "tags": ["lake"]}
```

Records carry their own keys; fields may be missing or nested.

Unstructured: text and media

„Утрото во Скопје беше студено и магливо, со температура околу два степени …“

image **audio** **video**

No predefined fields; structure must be extracted, for example by a language model.

- A *schema* defines field names, types and constraints. *Structured* data conform to one; *semi-structured* records carry keys and nesting (JSON, XML), with **optional validation**. *Unstructured* text and media have no predefined analytical fields; extracting those fields creates a structured representation.

Tabular data: observations, variables and dimension

NYC yellow-taxi trips, January 2025 — first 5 of 3,475,226 rows, 7 of 20 columns

pickup	PULocationID	DOLocationID	passengers	distance	payment	fare
2025-01-01 00:18	229	237	1	1.6	1	10.0
2025-01-01 00:32	236	237	1	0.5	1	5.1
2025-01-01 00:44	141	141	1	0.6	1	5.1
2025-01-01 00:14	244	244	3	0.52	2	7.2
2025-01-01 00:21	244	116	3	0.66	2	5.8

Observation (one row = one trip)

Dimension = number of variables (20)

Variable (one column = one type of measurement)

- In *tabular data* each row is an *observation* of **one type of object or event**, and each column is a *variable* (an attribute or feature); the number of variables is the *dimension*. Here every row is one taxi trip and the data set has 20 variables.

Quantitative and categorical variables

- A *quantitative* variable is numerical and supports arithmetic.
  - It is *continuous* when **any value in an interval** is possible, as for trip distance.
  - It is *discrete* when only **separated values** are possible, as for the number of passengers.
- A *categorical* variable takes values from a **set of labels**.
  - Payment type is coded as 0–5 in the file; the codes are labels and **must not be averaged**.
  - An *ordinal* variable has ordered labels, such as an education level.
- The figure uses **a histogram** for distance and bars for counts and labels; displayed ranges omit some outliers and rare codes.

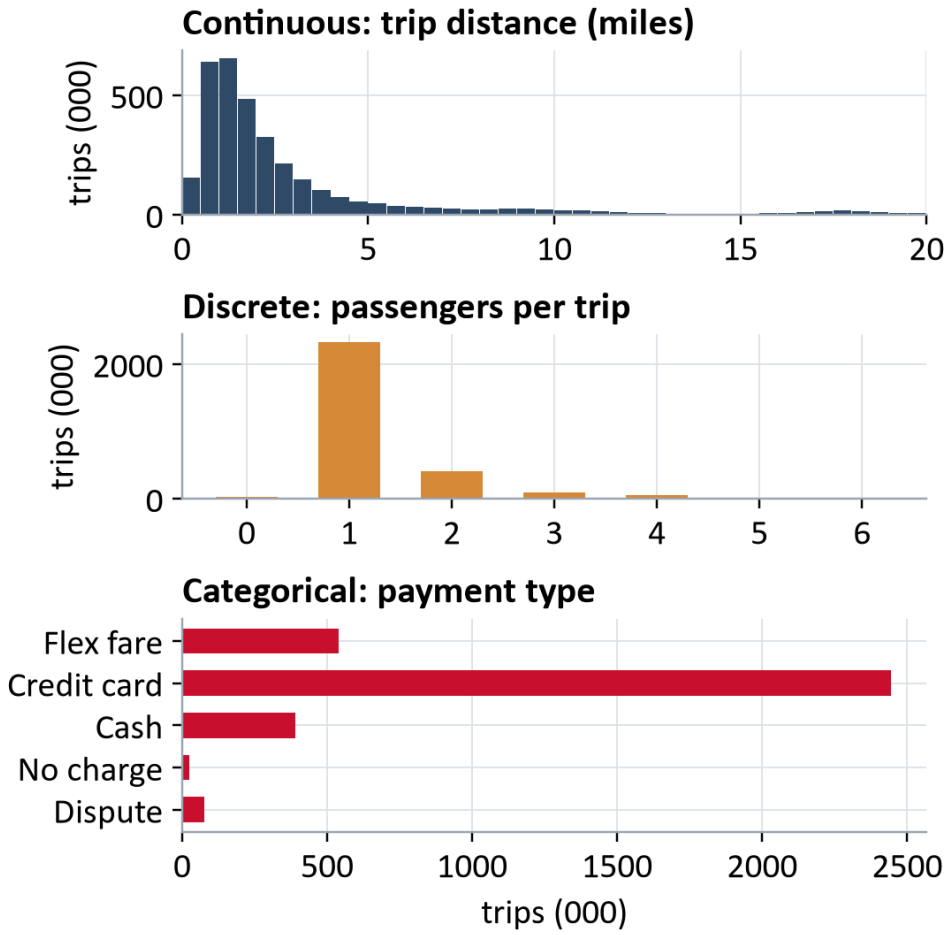


Figure 7 — three variables of the taxi data

Levels of measurement and meaningful operations

Level	Example	= ≠	< >	+ −	× ÷	Central value
Nominal	payment type, city	✓				mode
Ordinal	education level, rating	✓	✓			median
Interval	temperature in °C, year	✓	✓	✓		mean
Ratio	distance, duration, count	✓	✓	✓	✓	mean, ratios

Stevens (1946) four levels of measurement; meaningful operations depend on the measurement scale.

Categorical (qualitative) **Quantitative**

The table summarises meaningful comparisons, not every arithmetic operation. Celsius differences are meaningful, but 20 °C is not twice the temperature of 10 °C.

- The *level of measurement* of a variable decides **which operations and summaries are meaningful**: *nominal* values can only be compared for equality, *ordinal* values can also be ordered, *interval* values have meaningful differences, and *ratio* values have a true zero, so that ratios are meaningful.

Dimensions of data quality

- *Data quality* is the fitness of data **for a stated purpose**; it is assessed per dimension, not as a single score.
- A dimension is checked by a **rule that can be counted**: the share of missing values, of invalid codes or of duplicated keys.
- The dimensions conflict: a timely feed is often less complete than a later, corrected release.
- Quality problems found during acquisition are **recorded with the data**, so that later analysis can account for them.
  - This six-dimension set follows the UK Government Data Quality Framework (2020).

Completeness	Required values are present. *Example: passenger count missing*
Validity	Values conform to specified types and ranges. *Example: an undefined payment code*
Accuracy	Values correspond to the real world. *Example: a mistyped distance*
Consistency	Related values agree. *Example: drop-off before pick-up*
Timeliness	Data are recent enough for their purpose. *Example: a register updated yearly*
Uniqueness	Each entity is represented once. *Example: a trip stored twice*

Figure 8 — quality dimensions (UK, 2020)

Reshaping a reporting table into tidy form

Messy: 3 × 3 cells of counts

	Friday	Saturday	Sunday
Morning	15	158	10
Afternoon	2	90	20
Evening	55	12	45

```
tidy = (messy.rename_axis("Time")
    .reset_index().melt(
    id_vars="Time",
    var_name="Day",
    value_name="Number"))
```



Time	Day	Number
Morning	Friday	15
Afternoon	Friday	2
Evening	Friday	55
Morning	Saturday	158
Afternoon	Saturday	90
Evening	Saturday	12
Morning	Sunday	10
Afternoon	Sunday	20
Evening	Sunday	45

Tidy data (Wickham, 2014):
- each variable is a column;
- each observation is a row;
- each type of observational unit is a table.

- A *melt* turns **header values into a column**. Messy tables can also mix variables or observational units. Here Time is initially the row index; `reset_index` creates its column. `pivot` reverses this reshape only when each Time–Day pair is unique (Wickham, 2014).

03

Storage: formats, data models and architectures

Storage layouts, data models and architecture choices determine how the preceding tables are maintained and read.



Row-oriented and column-oriented storage

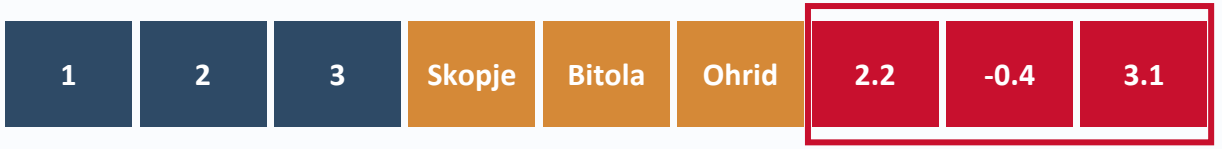
The logical table

id	city	temp
1	Skopje	2.2
2	Bitola	-0.4
3	Ohrid	3.1

Row-oriented file (CSV, JSON lines): one record after another



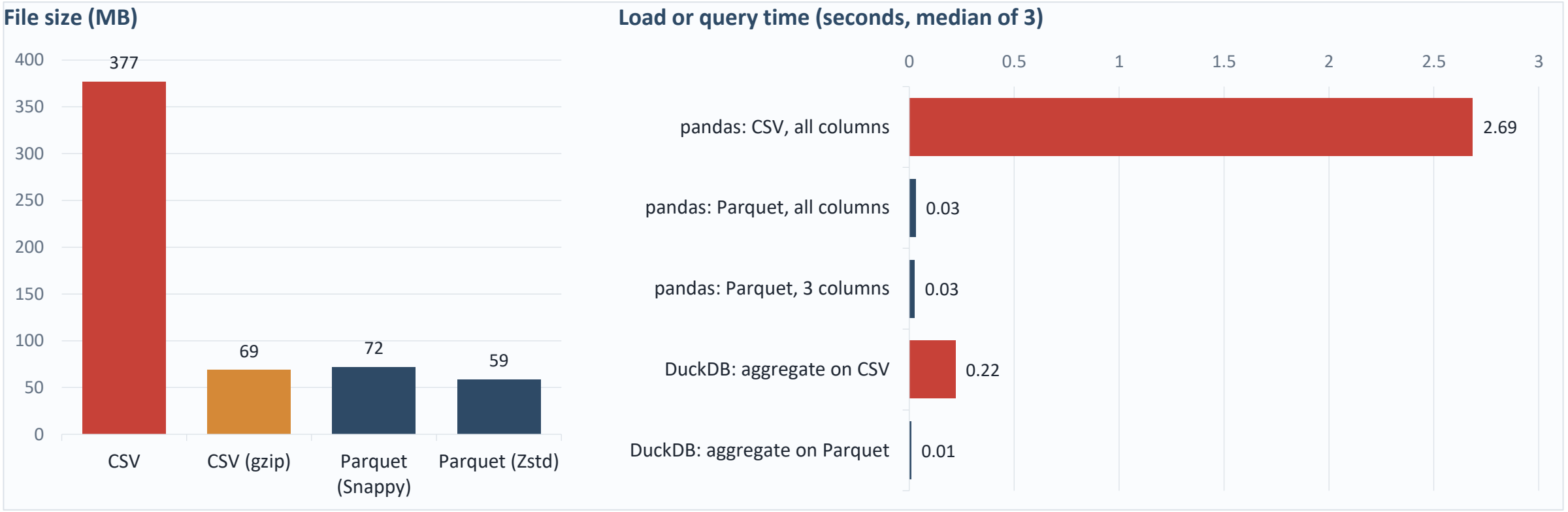
Column-oriented file (Parquet): values grouped within each row group



A query on temp alone reads one column chunk per row group (red frame) and can skip unrelated columns. This schematic shows one row group; similar values often compress well.

- A *row-oriented* format stores records sequentially, supporting **whole-record exchange**; a *column-oriented* format groups values by column, supporting **reads of selected columns**. CSV and JSON Lines are row-oriented text; Parquet is columnar binary. File formats alone do not provide database transactions.

CSV versus Parquet: file size and load time, measured



- The CSV file is **6.4 times larger** than Zstd-compressed Parquet (377 MB against 58 MB), and pandas loads it **91 times more slowly**. Gzip reduces size; gzip load time was not measured. Cached local experiment, 4 October 2026: three repetitions per operation, uncontrolled filesystem cache; ratios are not general guarantees.

04

Working with tables in pandas

Selecting, grouping and combining the tidy tables of the previous sections with pandas 3: masks, groupby, merge and concat



The pandas DataFrame: index, columns and data types

```
trips = pd.read_parquet("yellow_tripdata_2025-01.parquet")
```

index	tpep_pickup_datetime	PULocationID	fare_amount	store_and_fwd_flag
0	2025-01-01 00:18:38	229	10.0	N
1	2025-01-01 00:32:40	236	5.1	N
2	2025-01-01 00:44:04	141	5.1	N
…	…	…	…	…
dtype	datetime64[us]	int32	float64	str

Index: row labels (RangeIndex, 0 … 3,475,225)

3,475,226 rows × 20 columns; 4 of 20 columns shown. Each column holds one dtype.

pandas 3: text columns use str

Text is stored in the default *string dtype* `str`, backed by PyArrow when it is installed.

Missing text is NaN: 540,149 flags are missing.

pandas 3: Copy-on-Write

Every selection **behaves as a copy**; changing it never changes `trips`.

A change is written once on the original: `trips.loc[mask, "fare_amount"] = 0`.

- A *DataFrame* is a table of **labelled columns that share one index**; each column is a *Series* of a single dtype. `trips.shape`, `trips.dtypes` and `trips.info()` describe it; the January file occupies about 517 MB in memory. Copy-on-Write is treated in detail in the section on data cleaning.

Source: NYC TLC Yellow Taxi Trip Records, January 2025; pandas 3.0.6 documentation, Copy-on-Write and the string dtype (2026).

Selecting rows and columns: masks, .loc and .query

```
cols = ["tpep_pickup_datetime", "trip_distance",
        "fare_amount", "tip_amount"]
# zone 132 is JFK Airport; payment type 1 is a credit card
mask = (trips["PULocationID"] == 132) & (trips["payment_type"] == 1)
jfk = trips.loc[mask, cols]              # rows by mask, columns by label

same = trips.query("PULocationID == 132 and payment_type == 1")[cols]
jfk.equals(same), len(jfk)
jfk.head(3)
```

Result: (True, 101,313) and the first three rows

	tpep_pickup_datetime	trip_distance	fare_amount	tip_amount
86	2025-01-01 00:51:41	7.20	29.60	6.75
93	2025-01-01 00:55:44	14.84	59.00	12.30
215	2025-01-01 00:35:44	5.10	21.20	6.35

101,313 of 3,475,226 trips (2.9 %) are card-paid pick-ups at JFK Airport.

Source: NYC TLC Yellow Taxi Trip Records, January 2025; computed with pandas 3.

- A comparison on a column returns a *boolean mask*: one True or False **per row**, aligned on the index.
- Conditions are combined with &, | and ~; each comparison needs its **own parentheses**.
- `.loc[rows, columns]` selects **by label**: a mask for the rows, a list of names for the columns; `.iloc` selects by position.
- `.query()` writes the same filter as **one expression string**.
- The result keeps the **original row labels** (86, 93, 215), so every row can be traced back to the source table.

Grouping with groupby: trips and mean fare by hour

```
t = trips["tpep_pickup_datetime"]
jan = trips[(t >= "2025-01-01") & (t < "2025-02-01")
            & (trips["fare_amount"] > 0)]

by_hour = (jan.groupby(t.dt.hour.rename("pickup_hour"))
              .agg(trip_count=("fare_amount", "size"),
                   mean_fare=("fare_amount", "mean")))
by_hour.round(2)
```

Result: by_hour (24 rows, 4 shown)

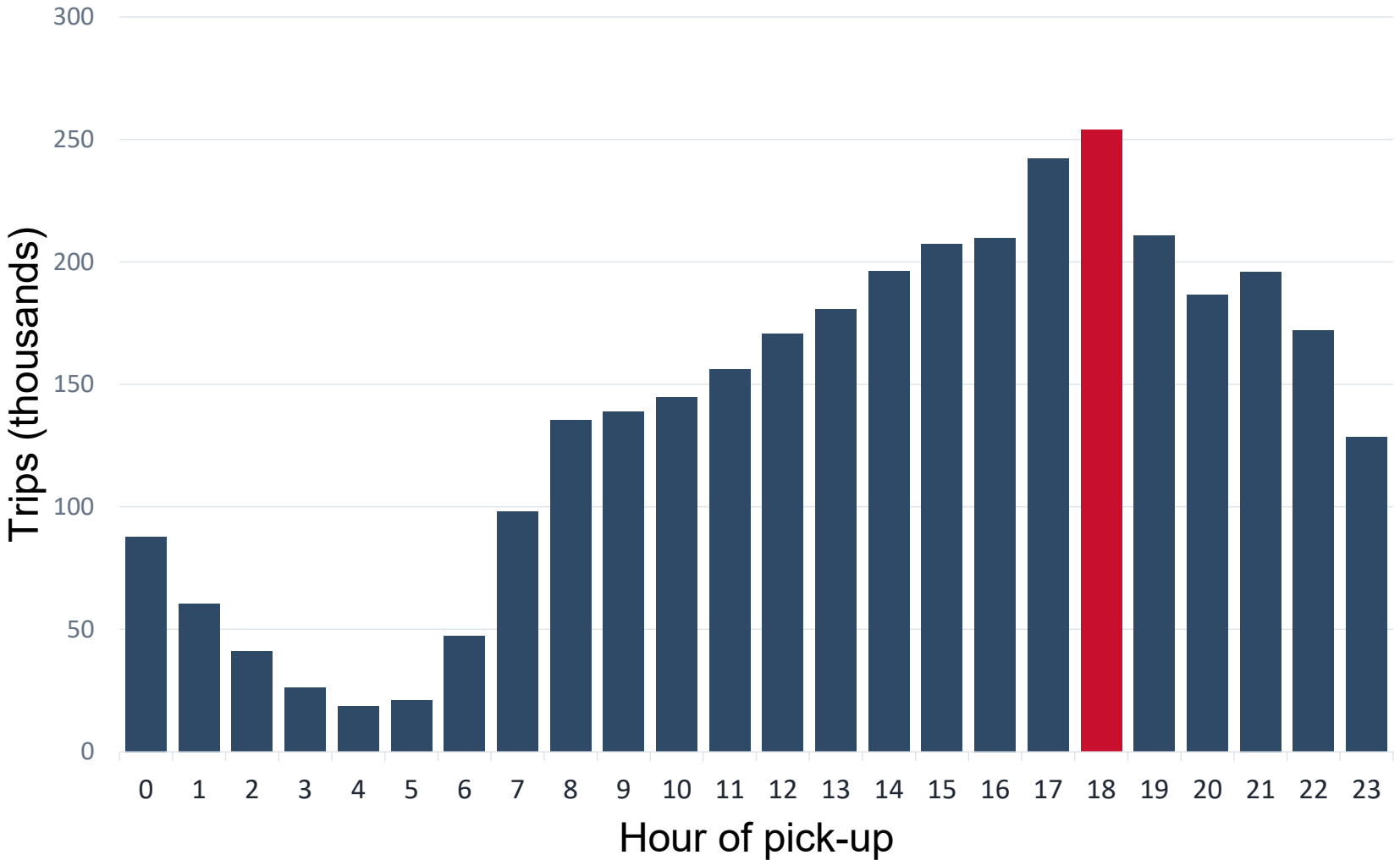
pickup_hour	trip_count	mean_fare
0	87,781	18.96
1	60,517	17.35
2	41,018	16.58
…	…	…
18	253,925	16.70

Peak hour 18:00 (red); the values equal the DS-02 aggregation.

Source: NYC TLC Yellow Taxi Trip Records, January 2025; computed with pandas 3.

- The mask keeps **January pick-ups with positive fares**; the half-open interval includes every January timestamp.
- `groupby` splits the rows by the **hour of the pick-up**, taken from the `.dt` accessor of the datetime column.
- `agg` applies **named summaries** to every group: `size` counts rows, `mean` skips missing fares.
- The result has **one row per group**: 24 rows indexed by `pickup_hour`, from 3,329,688 rows.
- The next two slides trace the three steps and plot the 24 counts.

Taxi trips by hour of the day, January 2025



- The preceding aggregation produces **24 hour groups**.
- Recorded trips peak at **18:00** with 254 thousand trips (red).
- The minimum count occurs at **4:00**, with 19 thousand trips.
- The mean fare is highest at 5:00 (**$25.70**); its cause is not established by this chart.
- Counts cover **positive-fare January pick-ups**, not all travel demand or unserved requests.

Combining tables in pandas: merge and concat

```
pd.merge(orders, regions, on="Client",
         how="left", indicator=True)
```

Client	Item	Month
John Doe	Coca-Cola	January
Jackie Qi	Pepsi-Cola	January
Ana Ilievska	Fanta	January

orders

Client	Region
John Doe	NY
Jackie Qi	NC

regions

Client	Item	Month	Region	_merge
John Doe	Coca-Cola	January	NY	both
Jackie Qi	Pepsi-Cola	January	NC	both
Ana Ilievska	Fanta	January	NaN	left_only

how="inner" keeps only the 2 matched rows; how="left" keeps Ana with NaN.

```
pd.concat([jan, feb], ignore_index=True)
```

	Client	Item	Month
0	John Doe	Coca-Cola	January
1	Jackie Qi	Pepsi-Cola	January
2	John Doe	Zero-Cola	February
3	Jackie Qi	Maxi-Cola	February

jan (2 rows)

feb (2 rows)

concat stacks frames along the rows and aligns them by column name; a column missing in one frame is filled with NaN. ignore_index=True renumbers the rows 0 … 3.

- A *merge* combines columns of two tables on **matching key values**; how= decides which unmatched rows survive, and `indicator=True` records the origin of every row in `_merge`. `concat` **appends rows** of frames with the same columns. The tables adapt Cielen, Meysman and Ali, Introducing Data Science (2016).

Source: computed with pandas 3; tables after Cielen, Meysman and Ali (2016).

Merging trips with the zone lookup: validate and indicator

```
zones = pd.read_csv("taxi_zone_lookup.csv")   # one row per zone
zones["LocationID"].is_unique

m = jan.merge(zones, how="left",
              left_on="PULocationID", right_on="LocationID",
              validate="many_to_one", indicator=True)
len(jan), len(m), m["_merge"].value_counts()

(m.groupby("Borough")["fare_amount"]
   .agg(["size", "mean"]).sort_values("size", ascending=False))
```

Result: True; (3,329,688, 3,329,688); both 3,329,688, left_only 0

Borough	size	mean fare
Manhattan	2,968,216	14.79
Queens	278,222	53.35
Brooklyn	59,523	26.60
Bronx	13,813	30.18

With zone 132 listed twice, the merge stops: MergeError: Merge keys are not unique in right dataset

Source: NYC TLC Yellow Taxi Trip Records, January 2025; computed with pandas 3.

- `validate="many_to_one"` checks that **every key is unique** in the lookup table before the merge runs.
  - A duplicated zone stops the merge with a `MergeError` instead of silently duplicating trips.
- A left merge of a many-to-one lookup **keeps the row count**: 3,329,688 rows before and after.
- `_merge` reports 3,329,688 matched and 0 unmatched trips.
- `groupby` drops missing keys: **1,285 trips** from zone 265 (Outside of NYC) have no borough.

Join cardinality and duplicated analytical measures

trip_id	fare
T1	10
T2	20

trip_id	tag
T1	airport
T1	card



trip_id	fare	tag
T1	10	airport
T1	10	card
T2	20	NULL

Illustrative left join: sum(fare) changes from 30 to 40 because the two matches duplicate T1.

- Join cardinality is determined by **matching-key multiplicity**. A many-to-one lookup requires a unique key in the lookup table; otherwise each match can duplicate measures. Key uniqueness, unmatched rows and row counts are checked before aggregation.

05

Data preparation and cleaning

Where preparation sits in the data science process of Lecture 1, and how raw tables are cleaned under pandas 3



Typical errors in raw tabular data

id	city	sex	age
1	␣Skopje	F	34
2	skopje	Female	150
3	Bitola␣	M	41
4		male	n/a
5	Ohrid	F	29

␣Skopje, Bitola␣	stray whitespace
skopje	case variant of one value
F, Female, male	one fact coded several ways
150	implausible age requiring verification
(empty)	missing value
n/a	text in a numerical column

Figure — error cells shaded red

- An *interpretation error* accepts an implausible value at face value, **such as an age of 150 years**.
- An *inconsistency error* records **one fact in several ways**: “F”, “Female” and “female”.
- Stray whitespace, case variants and entries such as “n/a” **prevent grouping and parsing**.
- A declared mapping merges known spelling variants through **bulk corrections**.
  - Each correction is stored as an operation history that can be replayed on new data.
- In a reproducible pipeline, the same corrections are written **as code**, applied in a fixed order.
  - A *codebook* lists the allowed values of each column, against which entries are checked.
- The codebook distinguishes **a valid category** from an unknown or inapplicable value; automated corrections must not merge genuinely different categories. A *schema* declares column names, types and nullability.

Copy-on-Write: copy and view semantics in pandas 3

- Version 3.0 of pandas (21 January 2026) makes *Copy-on-Write* **the only mode** of operation.
  - Every subset or derived object behaves as a copy; changing it never changes the parent.
  - The physical copy is **deferred until a write**, so reads stay cheap.
- *Chained assignment* indexes the table twice, first a column and then a subset of it; the write reaches **a temporary copy**, so the table itself does not change.
- A correct update selects **rows and column in one indexing step** on the table itself.
- Inferred text columns default to **a dedicated string type** rather than generic Python objects.
  - Code written for earlier versions, including code produced by AI assistants, is reviewed for chained assignment.
- A subset may already require a physical copy. **Data sharing is an optimisation**, not a promise; Copy-on-Write defines the observable mutation behaviour.

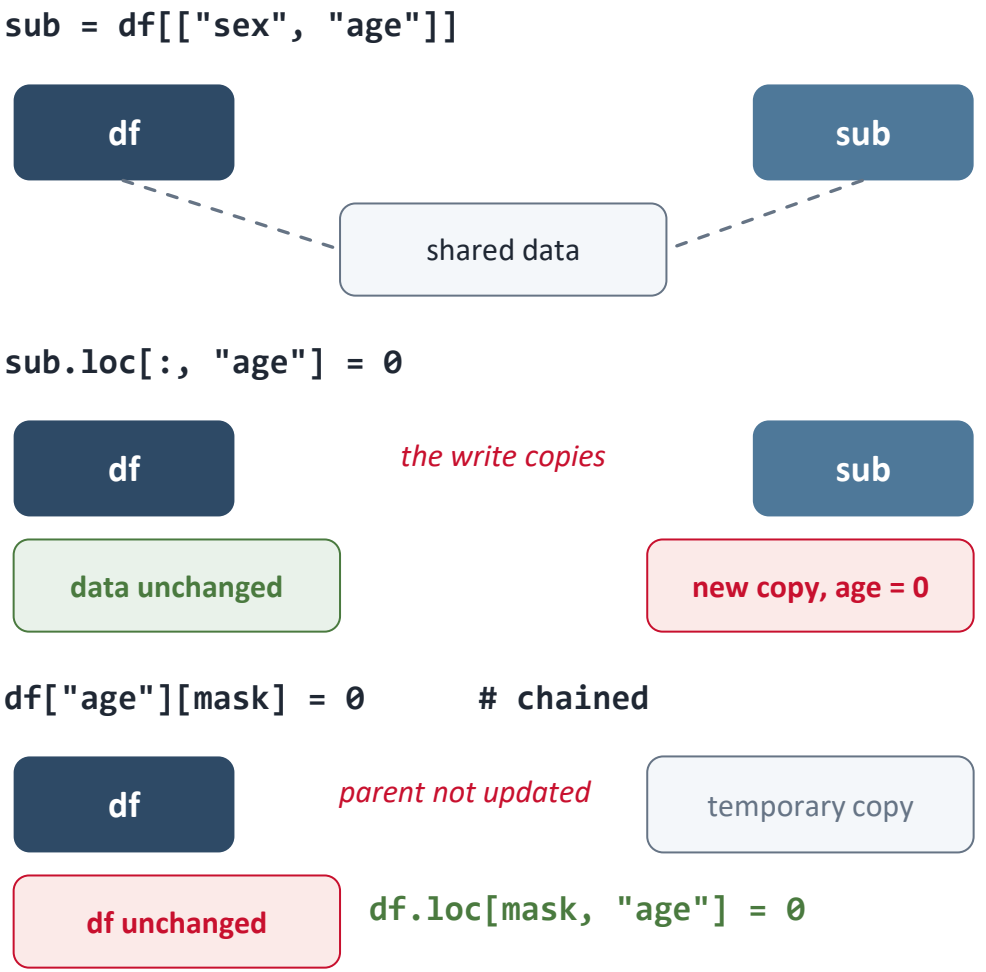


Figure — a write triggers the copy

Three mechanisms of missingness: MCAR, MAR and MNAR

Missing completely at random

- Under *MCAR*, missingness is **independent of observed and unobserved values**.
  - Example: an entry is lost when a coin lands heads.
- Complete cases form **a random subsample** under MCAR.
  - Their mean estimates the population mean without selection bias.
- Deletion loses information; mean imputation still **distorts variance**.
- The collection process must support **the independence assumption**.
- Valid inference still requires a suitable analysis model.

Missing at random

- Under *MAR*, missingness depends **only on observed variables**.
  - Example: younger respondents skip the income question more often.
- A complete-case mean can be biased because observed groups have **unequal representation**.
  - Appropriate imputation or weighting can address this under correct models and adequate overlap.
- MAR cannot be tested from the observed data alone.
- A subgroup with no observed incomes lacks **overlap**; imputation for it relies on extrapolation from other groups.

Missing not at random

- Under *MNAR*, missingness retains dependence **on unobserved values** after conditioning on observed variables.
  - Example: patients leave a trial because of an unmeasured side effect.
- Observed data alone do not identify **the missing distribution**.
  - Ordinary MAR imputation need not reduce the bias.
- External information and explicit assumptions support **sensitivity analysis**.
- A sensitivity analysis shifts assumed missing values and reports how substantive conclusions change.

Deletion and mean imputation: baseline methods

- *Listwise deletion* drops **every row that contains a gap**.
  - MCAR is sufficient for an unbiased complete-case mean; some **regression estimates** remain valid under weaker conditions.
- *Mean imputation* fills a numerical gap with **the mean of the observed values** of its column.
  - The most frequent value plays the same role for a categorical column.
- Mean imputation **shrinks the spread**: in the figure the standard deviation falls from 355 to 302.
  - Covariances and correlations are distorted; ordinary standard errors ignore imputation uncertainty.
- Both strategies are baselines against which **better imputers are measured**.
  - In a table with gaps in many columns, deletion can remove **most rows**, even when each column misses few values.
- An all-missing column has **no estimable mean**; retaining it requires an explicit constant, external information or removal.

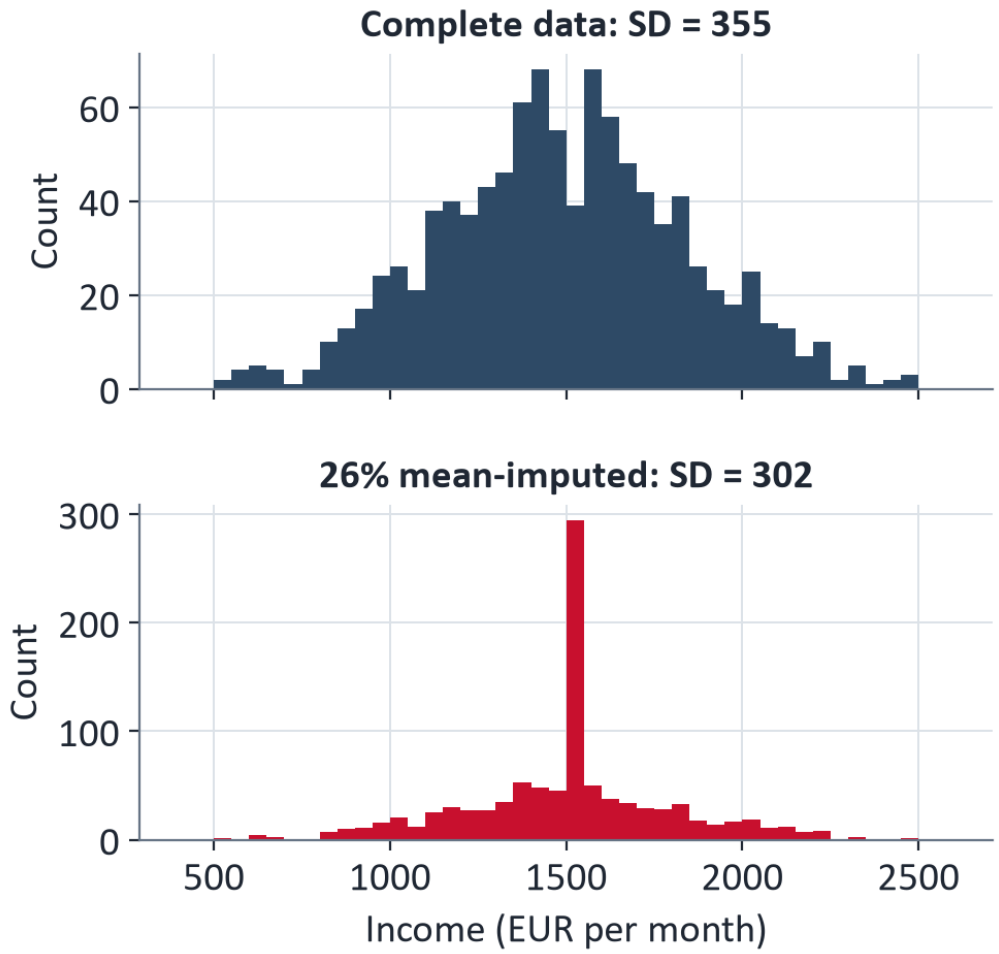


Figure — 26% filled with the column mean

A decision diagram for handling missing values

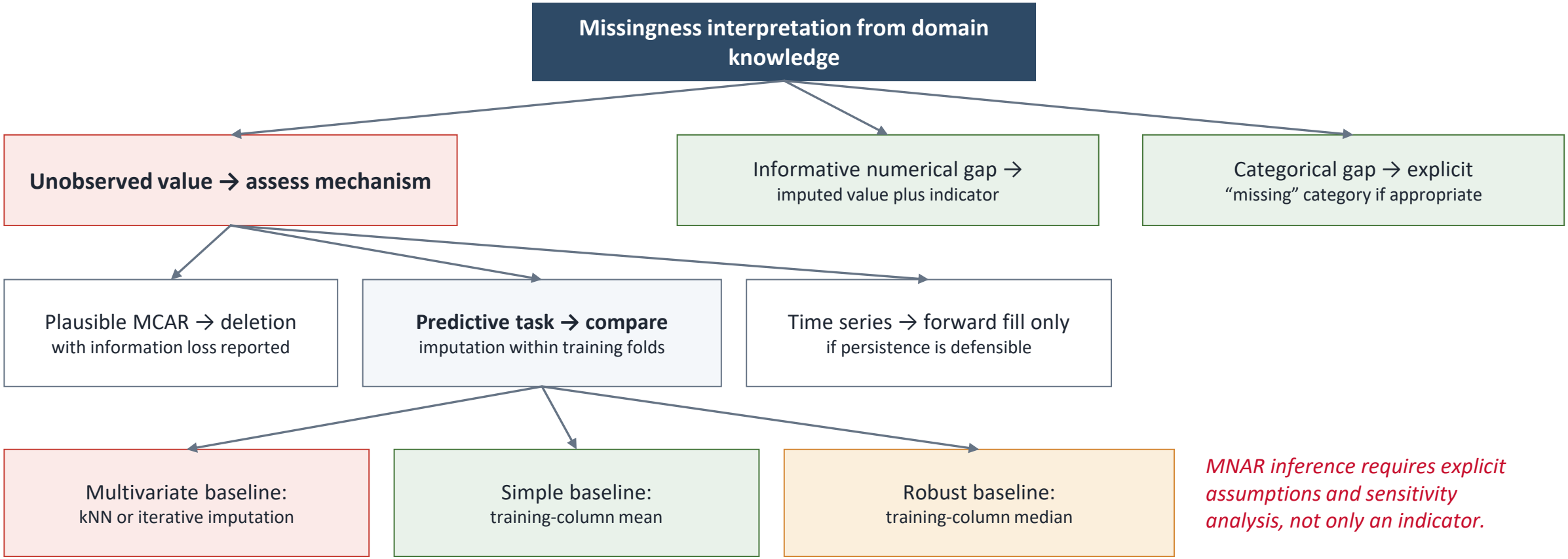


Figure — revised from Brink et al. (2016) using van Buuren (2018); method choice depends on task and assumptions

Detecting outliers with distribution plots and rules

- An *outlier* is an observation **distant from the others**, or one produced by a different process.
- Minima, maxima and **distribution plots** identify candidate univariate outliers, not every multivariate anomaly.
- The *three-sigma rule* flags values **more than three standard deviations** from the mean.
  - Extreme values inflate the standard deviation itself and can hide behind it.
- The *interquartile-range rule* flags values beyond **Q1 − 1.5·IQR and Q3 + 1.5·IQR**.
- On the 304 simulated values, the rules flag **4 and 5** points.
  - Isolation forests extend detection to many variables at once.
- Q1 and Q3 are the 25th and 75th percentiles; **IQR = Q3 − Q1**. A flagged point is a candidate for investigation, not proof of error.

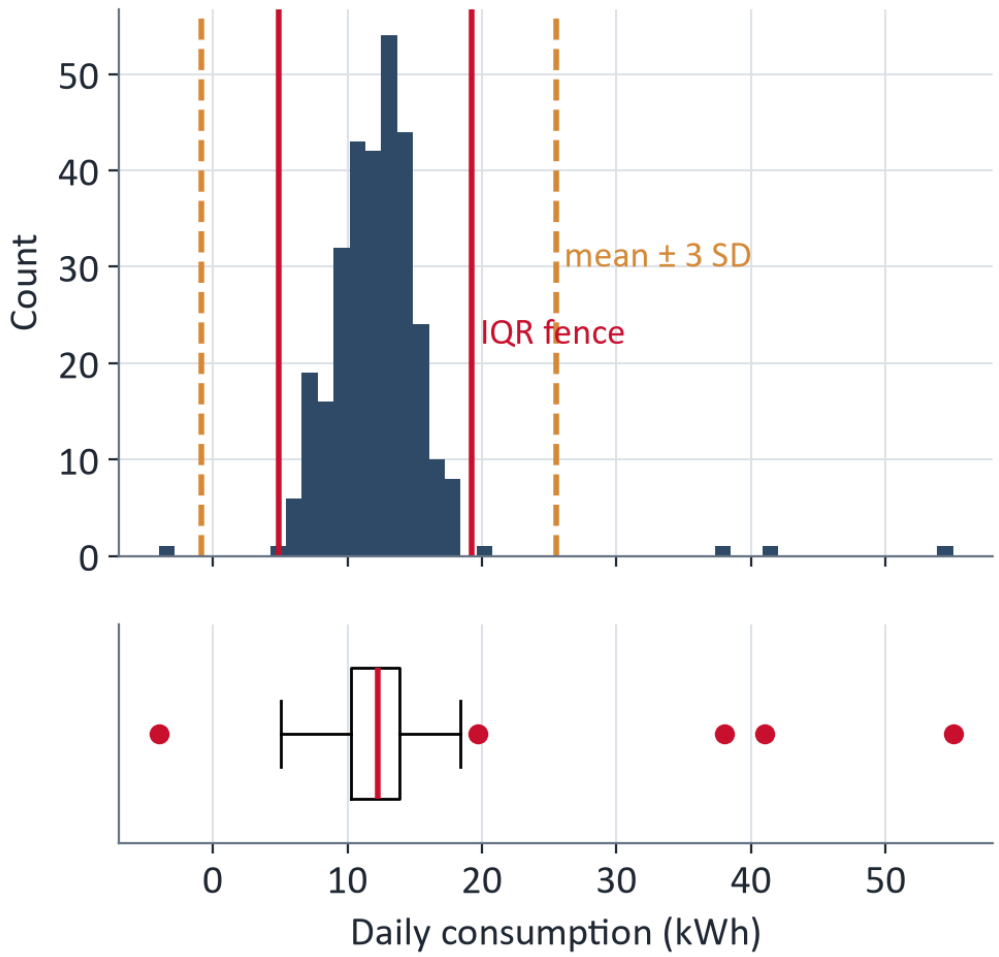


Figure — fences of the two rules

Outlier treatment: removal, transformation and capping

- *Removal* deletes the extreme value: 2, 3, 4, 5, 6, 10, 100 becomes **2, 3, 4, 5, 6, 10**.
- A *log transformation* compresses large values: $\log_{10}$ maps 100 to **2.00** and 10 to 1.00.
- *Capping* (winsorisation) replaces values above a threshold: at 10 the list ends **10, 10**.
- An outlier is **examined before it is treated**.
  - It may be a data error, a fraud case or the first member of a new cluster.
  - Anomalies can be the most informative rows of a data set.
- Thresholds such as the cap are learned **on the training data** and reused unchanged on new data.
  - Removal changes the sample size and is **reported** with the results.
- Capping changes the recorded distribution, whereas deleting rows changes who is represented. **Evaluation retains valid extreme cases** unless the intended population excludes them.

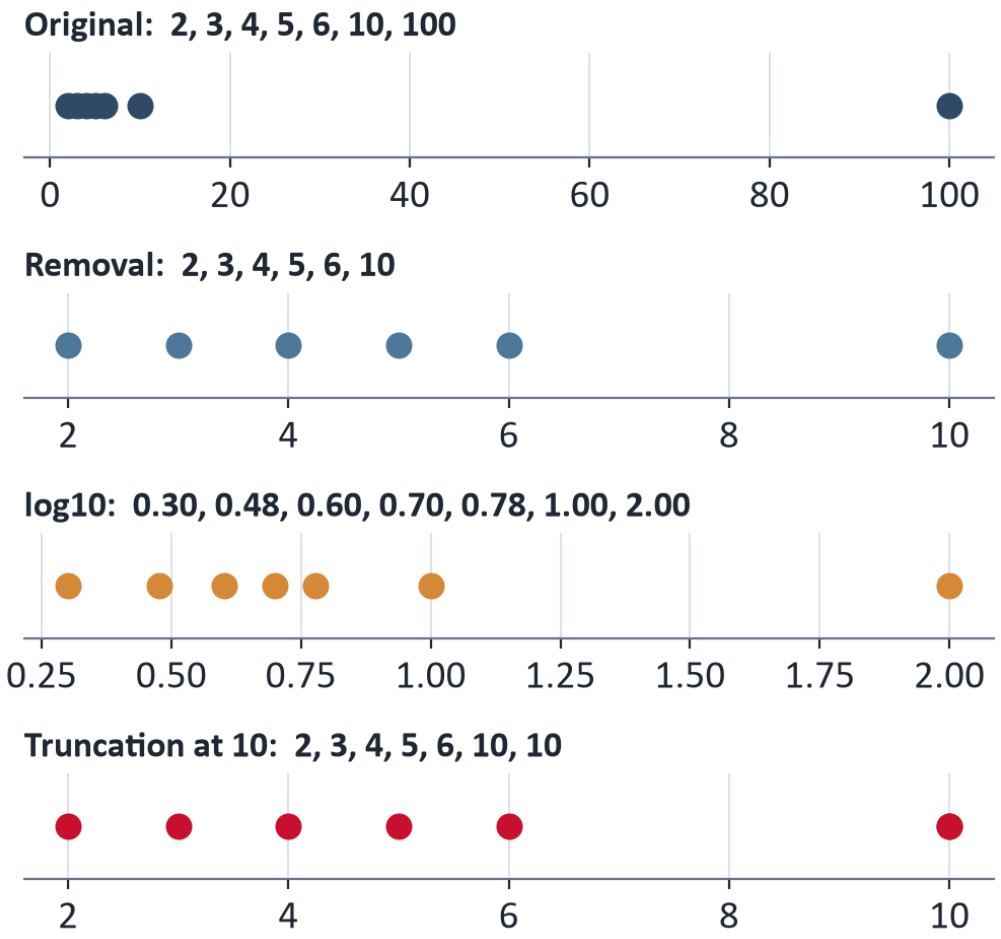


Figure — seven values, three treatments

Category encoding: integer, one-hot and binary codes

ONE-HOT

city	Sk	Bi	Oh	Te	Pr
Skopje	1	0	0	0	0
Bitola	0	1	0	0	0
Ohrid	0	0	1	0	0
Tetovo	0	0	0	1	0
Prilep	0	0	0	0	1

INTEGER

city	code
Skopje	0
Bitola	1
Ohrid	2
Tetovo	3
Prilep	4

BINARY

city	b2	b1	b0
Skopje	0	0	0
Bitola	0	0	1
Ohrid	0	1	0
Tetovo	0	1	1
Prilep	1	0	0

Figure — five cities in three encodings

- *Nominal* categories have no order; *ordinal* categories do: low < medium < high.
- *Integer encoding* maps each category to an integer 0 … n − 1.
  - Ordinal features need **a declared order**. Numeric tree splits also impose an order on nominal codes.
- *One-hot encoding* creates **one binary column per category**, with a 1 in the column of the observed category.
- *Binary encoding* writes zero-based codes in base 2, using **$\lceil \log_2 n \rceil$ columns** for n > 1.
  - Five cities need three bits; shared bits have no semantic meaning and introduce artificial similarities.
- High-cardinality columns, such as postal codes, make one-hot encoding **very wide**; frequency or target encoding is then used.
- `LabelEncoder` encodes targets in scikit-learn; `OrdinalEncoder` encodes feature columns. An **unknown-category policy** is specified before applying either mapping.

Proxy encoding and features from dates and times

- *Proxy encoding* replaces a category by **a meaningful number** associated with it.
  - A country by its GDP, a city by its coordinates, a category by its frequency.
- A timestamp is decomposed into **calendar and clock features**.
  - Year, month, weekday, weekend and holiday flags capture calendar effects.
  - Hour and seconds since midnight capture daily cycles.
- Elapsed time since a reference date turns a date into **one numerical feature**.
  - Cyclical features such as the hour are often encoded as sine and cosine, so that 23:00 lies close to 00:00.
- A date must use **the relevant time zone**; holidays and working hours depend on local calendars. Dates recorded after an outcome can reveal information unavailable at prediction time.

23-Oct-2023 20:39:18

feature	value
year	2023
month	10
day	23
day of week	Monday
day of year	296
weekend	0
hour	20
seconds since 00:00	74,358
working hours	0

Figure — one timestamp, nine features

Standardisation and normalisation of features

- Features with **larger numerical ranges** can dominate distance-based methods through scale alone.
  - A large numerical income difference can dominate an age difference in Euclidean distance.
- *Standardisation*: $z = (x - \mu) / \sigma$ gives **mean 0 and standard deviation 1**.
- *Min–max scaling* uses $x' = (x - \min) / (\max - \min)$ to map **training values to [0, 1]**.
  - One extreme value sets the range, so normalisation is sensitive to outliers.
- Standard threshold trees preserve splits under **strictly monotone rescaling**; distance-based and regularised models need not.
- The scaler is fitted on training rows; μ and σ are **reused unchanged** for test and production data.
- A constant feature has $\sigma = 0$; `StandardScaler` leaves it centred at zero using **scale 1** rather than dividing by zero.

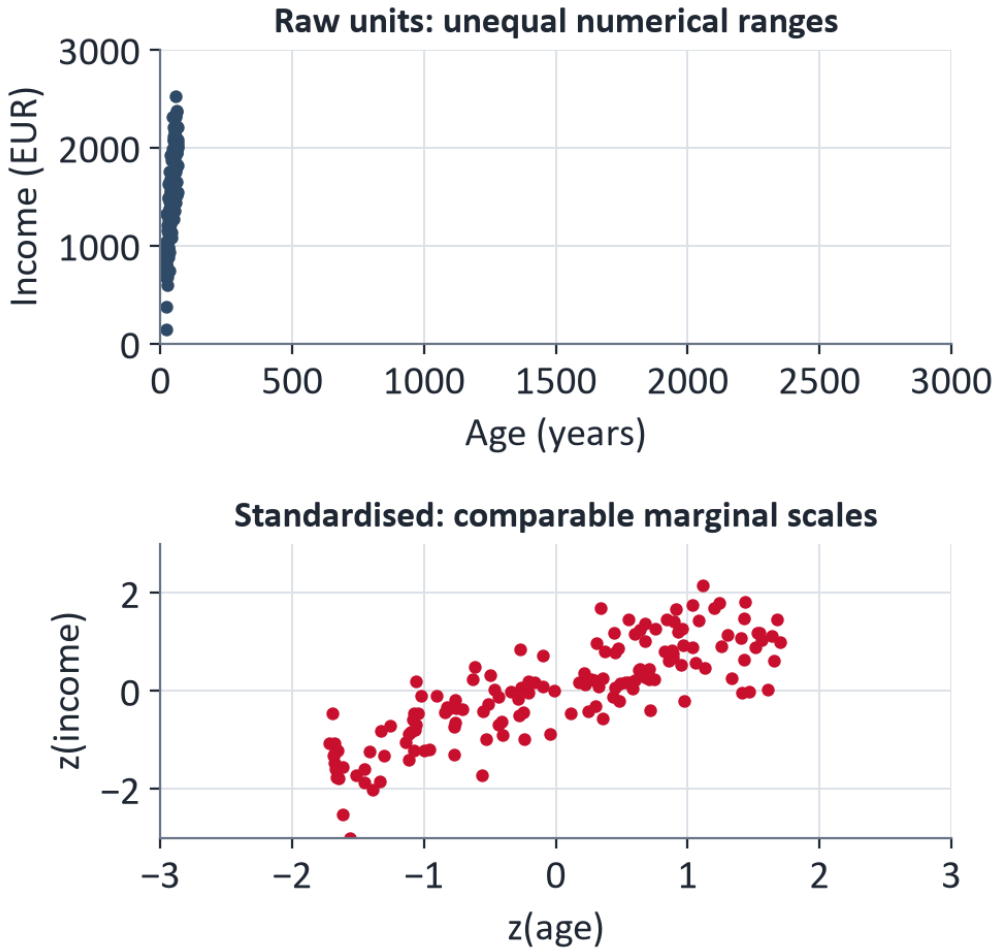


Figure — the same 150 points, two scales

06

Legal and ethical aspects of data

The rules that govern the acquisition of Section 01: personal data, anonymisation, licences and documentation.



Personal data under the GDPR and the Macedonian LPDP

> Personal data means any information relating to an identified or identifiable natural person, who can be identified directly or indirectly, in particular by a name, an identification number, location data or an online identifier.
>
> — After Regulation (EU) 2016/679 (GDPR), Article 4(1); the LPDP (Official Gazette 42/2020) adopts an equivalent definition

- A record is personal data when a person **can be singled out**, even without a name: a unique citizen number (EMBG), a licence plate or a precise trip start and end point.
  - The taxi file publishes zone identifiers rather than precise coordinates; this alone does not establish anonymity.

Data licences and the conditions they impose

Licence	Reuse and sharing	Attribution	Share-alike	Commercial use
CC0 1.0 / ODC-PDDL	Permitted	Not required	No	Permitted
CC BY 4.0	Permitted	Required	No	Permitted
ODC-ODbL 1.0	Permitted	Required	Yes, for the database	Permitted
CC BY-NC 4.0	Non-commercial only	Required	No	Not permitted
No licence stated	Rights must be checked	—	—	Basis required

Facts are not themselves copyrighted; compilations may attract copyright or database rights. EU Directive 2019/790, Article 4 concerns mining lawfully accessible material where rights are not expressly reserved; national implementation and Article 3 also matter. CC BY-NC is restricted, not an open-data licence. Privacy rights remain independent of the licence.

Table 1 — selected licences and restrictions; full terms and jurisdiction govern reuse

Key takeaways

- ✓ Data are acquired through **APIs and downloads first**; scraping is the last resort and has legal limits.
- ✓ The **level of measurement** decides which operations are meaningful; analysis expects **tidy tables**.
- ✓ **Columnar files** such as Parquet are smaller and faster than CSV.
- ✓ Grouping is **split–apply–combine**; the cardinality of a merge is **checked before measures are summed**.
- ✓ **Every cleaning step is code**, replayable on new data.
- ✓ The **missingness mechanism** determines which treatment is valid.
- ✓ Encoding and scaling turn raw columns into **features a model can use**.
- ✓ Personal data and licences are part of acquisition.



Questions

Co-funded by the European Union through the EuroHPC Joint Undertaking under Grant Agreement No 101263128, and by the Ministry of Digital Transformation of the Republic of North Macedonia. Views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the European Union or the EuroHPC JU.

Vezilka

AI Factories

EuroHPC Joint Undertaking

Co-funded by the European Union

Republic of North Macedonia
Ministry of Digital Transformation